

# Методичка: файловое хранилище и резервное копирование на Linux-сервере

**Темы:** VirtualBox, RAID1, mdadm, ext4, UUID, /etc/fstab, Linux-права, Samba, backup-скрипт, cron, восстановление данных

**Операционная система сервера:** Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

## 1. Что будет настроено

В этой работе будет собран небольшой файловый сервер file01 в VirtualBox.

Сервер получит:

- два виртуальных диска, объединённых в RAID1;
- файловую систему ext4, автоматически подключаемую в /srv/share;
- группу пользователей office и общие права на каталог;
- авторизованный сетевой ресурс Samba;
- отдельный виртуальный диск для резервных копий;
- backup-скрипт с архивированием, журналом и контрольной суммой;
- ежедневный запуск скрипта через cron;
- проверку восстановления удалённого файла;
- учебную проверку работы RAID1 при отказе одного диска.

Главная мысль работы проста:

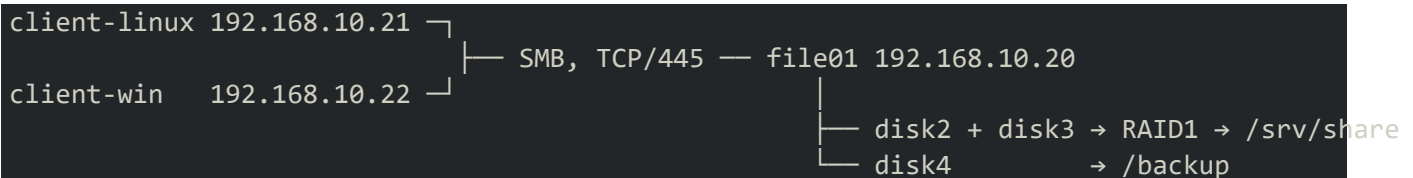
**RAID1 помогает серверу продолжить работу после отказа диска, а backup помогает вернуть удалённые или испорченные данные. Это разные уровни защиты.**

## 2. Схема учебного стенда

В VirtualBox создаются три виртуальные машины.

| Виртуальная машина | Назначение      | Операционная система                               | IP-адрес         |
|--------------------|-----------------|----------------------------------------------------|------------------|
| file01             | файловый сервер | Debian Server или Ubuntu Server                    | 192.168.10.20/24 |
| client-linux       | Linux-клиент    | Debian/Ubuntu с консолью или графической оболочкой | 192.168.10.21/24 |
| client-win         | Windows-клиент  | Windows 10/11                                      | 192.168.10.22/24 |

Логическая схема:



Для выполнения основной части достаточно file01 и одного клиента. Windows-клиент нужен для проверки доступа по UNC-пути. Если Windows-машины нет, ресурс можно полностью проверить с Linux-клиента через smbclient и CIFS-монтирование.

### 3. Подготовка виртуальных машин в VirtualBox

#### 3.1. Параметры сервера file01

Рекомендуемые ресурсы:

| Параметр           | Значение                    |
|--------------------|-----------------------------|
| Процессор          | 2 виртуальных ядра          |
| Оперативная память | 2–4 ГБ                      |
| Системный диск     | 20 ГБ                       |
| Диск RAID1 № 1     | 10 ГБ                       |
| Диск RAID1 № 2     | 10 ГБ                       |
| Диск для backup    | 15–20 ГБ                    |
| Сетевой адаптер 1  | NAT                         |
| Сетевой адаптер 2  | Внутренняя сеть storage-lab |

Адаптер NAT нужен для установки пакетов из репозитория. Внутренняя сеть используется для связи учебных машин между собой.

#### 3.2. Добавление виртуальных дисков

Виртуальная машина должна быть выключена.

В VirtualBox открывается:

Настроить → Носители → Контроллер SATA → Добавить жёсткий диск

Добавляются три новых диска:

1. raid-disk-1.vdi — 10 ГБ;
2. raid-disk-2.vdi — 10 ГБ;
3. backup-disk.vdi — 15–20 ГБ.

Для учебного стенда подходит формат VDI с динамическим выделением места.

После добавления дисков полезно создать снимок виртуальной машины:

Снимки → Создать → До настройки RAID и Samba

Снимок особенно пригодится перед проверкой отказа RAID1.

#### 3.3. Настройка внутренней сети

У всех трёх машин второй адаптер подключается к одной внутренней сети:

Тип подключения: Внутренняя сеть  
Имя: storage-lab

Имя сети должно совпадать символ в символ. Сети storage-lab и Storage-Lab для VirtualBox считаются разными.

### 4. Настройка IP-адресов

На сервере и клиентах нужно определить имя второго сетевого интерфейса.

`ip -br link`

## Что делает команда

`ip -br link` показывает сетевые интерфейсы в коротком формате. В VirtualBox они часто называются `enp0s3` и `enp0s8`, но имена могут отличаться.

## Пример ожидаемого вывода

```
lo                UNKNOWN    00:00:00:00:00:00
enp0s3            UP          08:00:27:11:22:33
enp0s8            UP          08:00:27:44:55:66
```

Обычно `enp0s3` связан с NAT, а `enp0s8` — с внутренней сетью. Это нужно подтвердить по настройкам VirtualBox и текущим адресам.

## 4.1. Вариант для Ubuntu Server с Netplan

Сначала студент смотрит имя файла Netplan:

```
ls /etc/netplan
```

Пример вывода:

```
50-cloud-init.yaml
```

Файл открывается редактором:

```
sudo nano /etc/netplan/50-cloud-init.yaml
```

Пример конфигурации сервера file01:

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      addresses:
        - 192.168.10.20/24
```

После сохранения конфигурация проверяется и применяется:

```
sudo netplan try
sudo netplan apply
ip -br address
```

`netplan try` временно применяет настройки и даёт возможность отменить их, если связь пропала.

Ожидаемая часть вывода `ip -br address`:

```
enp0s8            UP          192.168.10.20/24
```

## 4.2. Вариант для Debian Server с /etc/network/interfaces

Файл открывается редактором:

```
sudo nano /etc/network/interfaces
```

Для второго интерфейса добавляется:

```
auto enp0s8
iface enp0s8 inet static
    address 192.168.10.20/24
```

Настройки применяются перезапуском сетевой службы:

```
sudo systemctl restart networking  
ip -br address
```

Ожидаемая часть вывода:

```
enp0s8          UP          192.168.10.20/24
```

Если интерфейс называется не `enp0s8`, в конфигурации указывается фактическое имя из `ip -br link`.

### 4.3. Адреса клиентов

По той же схеме задаются адреса:

- `client-linux` — `192.168.10.21/24`;
- `client-win` — `192.168.10.22/24`.

Шлюз для внутренней сети не требуется. NAT-интерфейс сервера и Linux-клиента может получать адрес автоматически по DHCP.

### Самопроверка сети

С Linux-клиента:

```
ping -c 3 192.168.10.20
```

Ожидаемый результат:

```
3 packets transmitted, 3 received, 0% packet loss
```

С сервера:

```
ping -c 3 192.168.10.21
```

С Windows-клиента:

```
ping 192.168.10.20
```

Если ответы не приходят, сначала проверяются:

1. одинаковое имя внутренней сети в VirtualBox;
2. состояние второго адаптера;
3. IP-адрес и маска;
4. отсутствие ошибочно заданного шлюза на внутреннем интерфейсе.

---

## 5. Важное правило перед работой с дисками

Команды `mkfs`, `wipefs` и `mdadm --create` изменяют структуру диска. Если указать системный диск, виртуальная машина может перестать загрузаться.

В этой методичке используются имена:

```
/dev/sda — системный диск  
/dev/sdb — первый диск RAID1  
/dev/sdc — второй диск RAID1  
/dev/sdd — диск резервных копий
```

На конкретной машине имена могут быть другими. Поэтому студент не копирует команды вслепую, а сначала определяет назначение каждого диска.

## Часть 1. Создание RAID1 и файловой системы

### 6. Первичная диагностика сервера

#### 6.1. Проверка имени и версии системы

```
hostnamectl
cat /etc/os-release
```

##### Что делают команды

- `hostnamectl` показывает имя компьютера, ядро и архитектуру;
- `/etc/os-release` содержит название и версию дистрибутива.

Ожидаемый результат — сервер имеет имя `file01`, а в описании системы указана Debian или Ubuntu.

Пример:

```
Static hostname: file01
Operating System: Ubuntu 24.04.2 LTS
Kernel: Linux 6.8.0-xx-generic
Architecture: x86-64
```

Если имя сервера другое, его можно изменить:

```
sudo hostnamectl set-hostname file01
```

Проверка:

```
hostname
```

Ожидаемый вывод:

```
file01
```

#### 6.2. Проверка дисков

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS,UUID,MODEL
```

##### Что делает команда

`lsblk` показывает блочные устройства, их размер, файловую систему и точки монтирования. Это главный ориентир перед созданием RAID.

Пример ожидаемого вывода:

| NAME   | SIZE  | TYPE | FSTYPE | MOUNTPOINTS | UUID      | MODEL         |
|--------|-------|------|--------|-------------|-----------|---------------|
| sda    | 20G   | disk |        |             |           | VBOX HARDDISK |
| └─sda1 | 19G   | part | ext4   | /           | 1111-2222 |               |
| └─sda2 | 1G    | part | swap   | [SWAP]      | 3333-4444 |               |
| sdb    | 10G   | disk |        |             |           | VBOX HARDDISK |
| sdc    | 10G   | disk |        |             |           | VBOX HARDDISK |
| sdd    | 20G   | disk |        |             |           | VBOX HARDDISK |
| sr0    | 1024M | rom  |        |             |           | VBOX CD-ROM   |

В примере системный диск легко узнать по точке монтирования `/`. Диски `sdb`, `sdc` и `sdd` пока не имеют файловой системы и не смонтированы.

Дополнительная безопасная проверка сигнатур:

```
sudo wipefs -n /dev/sdb
sudo wipefs -n /dev/sdc
sudo wipefs -n /dev/sdd
```

### Что делает команда

Ключ -n включает режим просмотра. Команда ничего не стирает, а только показывает найденные сигнатуры файловых систем или RAID.

Для новых дисков вывод обычно пустой. Если команда показывает ext4, dos, gpt или linux\_raid\_member, диск уже использовался. В этом случае студент ещё раз проверяет, что выбран именно учебный диск.

### Самопроверка этапа

Перед продолжением должны выполняться все условия:

- системная файловая система находится на отдельном диске;
- два диска RAID имеют одинаковый размер;
- backup-диск не является системным;
- на учебных дисках нет точек монтирования.

## 7. Установка пакетов

```
sudo apt update
sudo apt install -y mdadm samba smbclient cifs-utils acl cron
```

### Что устанавливается

| Пакет      | Назначение                                         |
|------------|----------------------------------------------------|
| mdadm      | создание и обслуживание программного RAID          |
| samba      | сервер SMB                                         |
| smbclient  | консольная проверка SMB-ресурсов                   |
| cifs-utils | подключение SMB-ресурса как файловой системы Linux |
| acl        | просмотр и настройка расширенных прав              |
| cron       | запуск задач по расписанию                         |

Во время установки mdadm Debian может спросить, нужно ли запускать проверку RAID автоматически. Для учебного стенда можно оставить предложенное значение по умолчанию.

### Проверка установки

```
mdadm --version
smbd --version
smbclient --version
```

Пример ожидаемого вывода:

```
mdadm - v4.x
Version 4.x.x
Version 4.x.x
```

Номера версий могут отличаться. Важно, чтобы команды запускались без сообщения command not found.

---

## 8. Создание RAID1

В учебной работе RAID создаётся из целых виртуальных дисков. В реальной инфраструктуре часто используют разделы /dev/sdb1 и /dev/sdc1, но для понимания механизма это не обязательно.

Команда создания массива:

```
sudo mdadm --create /dev/md0 \  
  --level=1 \  
  --raid-devices=2 \  
  /dev/sdb /dev/sdc
```

### Что означает команда

- /dev/md0 — имя нового программного RAID-устройства;
- --level=1 — зеркалирование RAID1;
- --raid-devices=2 — массив состоит из двух дисков;
- /dev/sdb /dev/sdc — участники массива.

mdadm может предупредить, что на дисках будут записаны метаданные, и запросить подтверждение:

```
Continue creating array? y
```

После ввода y начинается синхронизация.

### Проверка состояния RAID

```
cat /proc/mdstat
```

Пример вывода во время синхронизации:

```
Personalities : [raid1]  
md0 : active raid1 sdc[1] sdb[0]  
      10475520 blocks super 1.2 [2/2] [UU]  
      [===>.....] resync = 24.7% finish=1.2min speed=102400K/sec
```

Обозначение [UU] показывает, что оба участника массива активны.

Для наблюдения за синхронизацией:

```
watch -n 2 cat /proc/mdstat
```

Выход из watch выполняется сочетанием Ctrl+C.

Подробная информация:

```
sudo mdadm --detail /dev/md0
```

Ожидаемые строки:

```
Raid Level : raid1  
Raid Devices : 2  
Total Devices : 2  
State : clean  
Active Devices : 2  
Working Devices : 2  
Failed Devices : 0
```

Во время первой синхронизации состояние может быть clean, resyncing. Это нормально.

## Самопроверка этапа

```
cat /proc/mdstat  
sudo mdadm --detail /dev/md0 | grep -E 'Raid Level|State|Active Devices|Failed Devices'
```

Успешный результат:

- массив /dev/md0 существует;
  - уровень — raid1;
  - активны два диска;
  - отсутствуют failed-устройства;
  - индикатор массива — [UU].
- 

## 9. Сохранение конфигурации RAID

Debian и Ubuntu обычно умеют автоматически находить RAID-массив, но его описание лучше сохранить в конфигурации mdadm.

Сначала команда выводит строку массива:

```
sudo mdadm --detail --scan
```

Пример:

```
ARRAY /dev/md0 metadata=1.2 UUID=8c90bde5:1a2b3c4d:5e6f7788:9abcde01
```

Файл конфигурации открывается:

```
sudo nano /etc/mdadm/mdadm.conf
```

В конец файла добавляется строка ARRAY, полученная на предыдущем шаге. Если похожая строка уже есть, вторую добавлять не нужно.

После сохранения обновляется начальный загрузочный образ:

```
sudo update-initramfs -u
```

Пример нормального вывода:

```
update-initramfs: Generating /boot/initrd.img-6.x.x
```

## Проверка после перезагрузки

```
sudo reboot
```

После запуска системы:

```
cat /proc/mdstat  
sudo mdadm --detail /dev/md0
```

Ожидаемый результат — массив снова найден, а состояние остаётся [UU].

Если /dev/md0 не появился, студент не создаёт массив заново. Сначала выполняется поиск существующего массива:

```
sudo mdadm --assemble --scan
```

---

## 10. Создание файловой системы и первое монтирование

На RAID-устройстве создаётся файловая система ext4:



```
sudo mkfs.ext4 -L SHARE_RAID /dev/md0
```

### Что делает команда

- `mkfs.ext4` создаёт файловую систему `ext4`;
- `-L SHARE_RAID` задаёт понятную метку;
- `/dev/md0` — устройство, которое форматируется.

Фрагмент ожидаемого вывода:

```
Creating filesystem with ... blocks and ... inodes
Filesystem UUID: 12345678-abcd-4321-9876-123456789abc
Writing superblocks and filesystem accounting information: done
```

Создаётся точка монтирования:

```
sudo mkdir -p /srv/share
```

Массив подключается вручную:

```
sudo mount /dev/md0 /srv/share
```

Проверка:

```
findmnt /srv/share
df -hT /srv/share
```

Пример вывода `findmnt`:

| TARGET     | SOURCE   | FSTYPE | OPTIONS     |
|------------|----------|--------|-------------|
| /srv/share | /dev/md0 | ext4   | rw,relatime |

Пример вывода `df`:

| Filesystem | Type | Size | Used | Avail | Use% | Mounted on |
|------------|------|------|------|-------|------|------------|
| /dev/md0   | ext4 | 9.8G | 24K  | 9.3G  | 1%   | /srv/share |

### Самопроверка этапа

```
mountpoint /srv/share
```

Ожидаемый вывод:

```
/srv/share is a mountpoint
```

---

## 11. Автомонтирование RAID по UUID

Имя `/dev/md0` обычно стабильно, но в `/etc/fstab` лучше использовать UUID файловой системы. UUID относится к самой файловой системе и не зависит от порядка обнаружения дисков.

### 11.1. Получение UUID

```
sudo blkid /dev/md0
```

Пример:

```
/dev/md0: LABEL="SHARE_RAID" UUID="12345678-abcd-4321-9876-123456789abc" BLOCK_SIZE="4096" TYPE=
```

Также можно использовать:

```
lsblk -f
```

## 11.2. Резервная копия /etc/fstab

```
sudo cp /etc/fstab /etc/fstab.bak
```

Проверка создания копии:

```
ls -l /etc/fstab /etc/fstab.bak
```

## 11.3. Добавление записи

Файл открывается:

```
sudo nano /etc/fstab
```

В конец добавляется строка с фактическим UUID:

```
UUID=12345678-abcd-4321-9876-123456789abc /srv/share ext4 defaults 0 2
```

Поля означают:

1. UUID файловой системы;
2. точка монтирования;
3. тип файловой системы;
4. параметры монтирования;
5. резервный параметр dump;
6. порядок проверки fsck.

## 11.4. Проверка без перезагрузки

Сначала ресурс отключается:

```
sudo umount /srv/share
```

Проверяется синтаксис и логика fstab:

```
sudo findmnt --verify --verbose
```

Успешный результат обычно заканчивается сообщением:

```
Success, no errors or warnings detected
```

Затем выполняется подключение всех записей:

```
sudo mount -a
```

Если команда ничего не вывела, это обычно означает отсутствие ошибок.

Итоговая проверка:

```
findmnt /srv/share  
mountpoint /srv/share
```

Ожидаемый результат:

```
TARGET    SOURCE    FSTYPE OPTIONS  
/srv/share /dev/md0 ext4    rw,relatime  
/srv/share is a mountpoint
```

Если `mount -a` сообщает об ошибке, сервер пока не перезагружается. Сначала исправляется запись или возвращается резервная копия: `sudo cp /etc/fstab.bak /etc/fstab`.

## 12. Подготовка отдельного диска для резервных копий

Backup не должен лежать на том же RAID1, который он защищает. В учебном стенде для него используется третий виртуальный диск.

Файловая система создаётся на /dev/sdd:

```
sudo mkfs.ext4 -L BACKUP /dev/sdd
```

Создаётся каталог:

```
sudo mkdir -p /backup
```

Диск подключается:

```
sudo mount /dev/sdd /backup
```

Проверка:

```
findmnt /backup  
df -hT /backup
```

Ожидаемый пример:

```
TARGET SOURCE FSTYPE OPTIONS  
/backup /dev/sdd ext4 rw,relatime
```

Получается UUID:

```
sudo blkid /dev/sdd
```

Пример:

```
/dev/sdd: LABEL="BACKUP" UUID="87654321-dcba-1234-5678-abcdef123456" TYPE="ext4"
```

В /etc/fstab добавляется строка:

```
UUID=87654321-dcba-1234-5678-abcdef123456 /backup ext4 defaults,nofail 0 2
```

Параметр nofail позволяет системе загрузиться, даже если backup-диск временно отсутствует. Сам backup-скрипт позже отдельно проверит, что /backup действительно подключён.

Проверка:

```
sudo umount /backup  
sudo findmnt --verify --verbose  
sudo mount -a  
findmnt /backup
```

### Самопроверка двух файловых систем

```
findmnt --target /srv/share  
findmnt --target /backup
```

Ожидаемый смысл результата:

- /srv/share расположен на /dev/md0;
  - /backup расположен на отдельном /dev/sdd;
  - обе файловые системы имеют тип ext4.
-

## Часть 2. Пользователи, права и Samba

### 13. Создание группы и пользователей

Общий ресурс будет доступен членам группы office.

Создание группы:

```
sudo groupadd office
```

Если группа уже существует, система выведет:

```
groupadd: group 'office' already exists
```

Это не проблема. Повторно создавать группу не нужно.

Создание двух пользователей без домашнего каталога и интерактивной оболочки:

```
sudo useradd -M -s /usr/sbin/nologin -G office ivanov
sudo useradd -M -s /usr/sbin/nologin -G office petrova
```

#### Что означают параметры

- -M — не создавать домашний каталог;
- -s /usr/sbin/nologin — запретить обычный вход в shell;
- -G office — добавить пользователя в дополнительную группу office.

Проверка:

```
getent group office
id ivanov
id petrova
```

Пример ожидаемого вывода:

```
office:x:1001:ivanov,petrova
uid=1001(ivanov) gid=1002(ivanov) groups=1002(ivanov),1001(office)
uid=1002(petrova) gid=1003(petrova) groups=1003(petrova),1001(office)
```

Если пользователь уже существует, он добавляется в группу так:

```
sudo usermod -aG office ivanov
```

Ключ -a важен: без него можно случайно заменить список дополнительных групп пользователя.

---

### 14. Настройка Linux-прав каталога

Сначала каталогу назначаются владелец и группа:

```
sudo chown root:office /srv/share
```

Затем устанавливаются права:

```
sudo chmod 2770 /srv/share
```

#### Что означает 2770

- первая 2 включает setgid для каталога;
- владелец получает rwx;
- группа получает rwx;
- остальные пользователи не получают доступа.

Setgid нужен для того, чтобы новые файлы наследовали группу office, а не основную группу автора.

Настраиваются наследуемые ACL:

```
sudo setfacl -m d:u::rwx,d:g::rwx,d:m::rwx,d:o:--- /srv/share
```

Проверка:

```
ls -ld /srv/share
getfacl /srv/share
```

Пример `ls -ld`:

```
drwxrws---+ 2 root office 4096 Jul 16 12:00 /srv/share
```

Здесь:

- s в позиции прав группы показывает setgid;
- + в конце режима показывает наличие ACL.

Фрагмент ожидаемого вывода `getfacl`:

```
# owner: root
# group: office
user::rwx
group::rwx
mask::rwx
other:---
default:user::rwx
default:group::rwx
default:mask::rwx
default:other:---
```

## Проверка прав без Samba

Пользователь `ivanov` создаёт файл:

```
sudo -u ivanov bash -c 'echo "created by ivanov" > /srv/share/linux-rights-test.txt'
```

Пользователь `petrova` читает файл:

```
sudo -u petrova cat /srv/share/linux-rights-test.txt
```

Ожидаемый вывод:

```
created by ivanov
```

Проверка владельца и группы:

```
ls -l /srv/share/linux-rights-test.txt
```

Пример:

```
-rw-rw---+ 1 ivanov office 18 Jul 16 12:05 /srv/share/linux-rights-test.txt
```

Если возникает `Permission denied`, Samba пока не настраивается. Сначала проверяются:

```
id ivanov
id petrova
ls -ld /srv/share
getfacl /srv/share
```

## 15. Создание учётных записей Samba

Linux-пользователь и Samba-пользователь связаны, но пароль Samba хранится отдельно.

Добавление пользователей в базу Samba:

```
sudo smbpasswd -a ivanov
sudo smbpasswd -a petrova
```

Команда запросит новый пароль дважды:

```
New SMB password:
Retype new SMB password:
Added user ivanov.
```

Учётные записи включаются:

```
sudo smbpasswd -e ivanov
sudo smbpasswd -e petrova
```

Проверка базы Samba:

```
sudo pdbedit -L
```

Пример:

```
ivanov:1001:
petrova:1002:
```

UID могут отличаться.

---

## 16. Настройка общего ресурса Samba

Сначала создаётся резервная копия конфигурации:

```
sudo cp /etc/samba/smb.conf /etc/samba/smb.conf.bak.$(date +%F-%H%M)
```

Файл открывается:

```
sudo nano /etc/samba/smb.conf
```

В конец добавляется секция:

```
[share]
    comment = Учебный общий ресурс
    path = /srv/share
    browsable = yes
    read only = no
    guest ok = no
    valid users = @office
    force group = office
    create mask = 0660
    force create mode = 0660
    directory mask = 2770
    force directory mode = 2770
```

Что означают параметры

| Параметр              | Назначение                                 |
|-----------------------|--------------------------------------------|
| [share]               | сетевое имя ресурса                        |
| path                  | каталог Linux, который публикуется         |
| browsable             | ресурс виден в списке общих папок          |
| read only = no        | запись разрешена на уровне Samba           |
| guest ok = no         | анонимный доступ запрещён                  |
| valid users = @office | подключаться могут члены группы office     |
| force group = office  | создаваемые объекты получают группу office |
| create mask           | максимальные права новых файлов            |
| directory mask        | максимальные права новых каталогов         |

### 16.1. Проверка синтаксиса

```
sudo testparm -s
```

В норме команда заканчивается примерно так:

```
Loaded services file OK.
Server role: ROLE_STANDALONE
```

В выводе должна присутствовать секция [share].

Если testparm показывает ошибку, служба пока не перезапускается. Сначала исправляется указанная строка.

### 16.2. Запуск Samba

```
sudo systemctl enable --now smbd
sudo systemctl restart smbd
sudo systemctl status smbd --no-pager
```

Ожидаемая строка:

```
Active: active (running)
```

Проверка порта SMB:

```
sudo ss -lntp | grep ':445'
```

Пример:

```
LISTEN 0 50 0.0.0.0:445 0.0.0.0:* users:(("smbd",pid=1234,fd=31))
LISTEN 0 50 [::]:445 [::]:* users:(("smbd",pid=1234,fd=29))
```

### 16.3. Проверка firewall

На чистой Debian firewall обычно не включён. На Ubuntu может использоваться UFW.

Проверка:

```
sudo ufw status
```

Если команда сообщает Status: active, разрешается профиль Samba:

```
sudo ufw allow Samba
sudo ufw status
```

Ожидаемая запись:

|       |       |          |
|-------|-------|----------|
| Samba | ALLOW | Anywhere |
|-------|-------|----------|

Если команда `ufw` не установлена или `firewall` неактивен, этот шаг пропускается.

---

## 17. Локальная проверка Samba на сервере

Список ресурсов:

```
smbclient -L //localhost -U ivanov
```

После ввода пароля ожидается таблица с ресурсом share:

| Sharename | Type | Comment              |
|-----------|------|----------------------|
| -----     | ---- | -----                |
| share     | Disk | Учебный общий ресурс |
| IPC\$     | IPC  | IPC Service          |

Подключение к ресурсу:

```
smbclient //localhost/share -U ivanov
```

После успешного входа появляется приглашение:

```
smb: \>
```

Внутри `smbclient` выполняются команды:

```
ls
put /etc/hostname hostname-from-smbclient.txt
ls
exit
```

Ожидаемый результат загрузки:

```
putting file /etc/hostname as \hostname-from-smbclient.txt
```

Проверка на сервере:

```
ls -l /srv/share
```

Файл должен иметь группу `office`:

```
-rw-rw----+ 1 ivanov office ... hostname-from-smbclient.txt
```

---

## 18. Проверка с Linux-клиента

На Linux-клиенте устанавливаются инструменты:

```
sudo apt update
sudo apt install -y smbclient cifs-utils
```

Проверяется доступность сервера:

```
ping -c 3 192.168.10.20
```

Проверяется список ресурсов:

```
smbclient -L //192.168.10.20 -U ivanov
```

Создаётся точка монтирования:



```
sudo mkdir -p /mnt/share
```

Ресурс подключается:

```
sudo mount -t cifs //192.168.10.20/share /mnt/share \
-o username=ivanov,vers=3.0
```

Команда запросит пароль Samba.

Проверка монтирования:

```
findmnt /mnt/share
```

Пример ожидаемого вывода:

| TARGET     | SOURCE                | FSTYPE | OPTIONS                  |
|------------|-----------------------|--------|--------------------------|
| /mnt/share | //192.168.10.20/share | cifs   | rw,relatime,vers=3.0,... |

Создание файла:

```
echo "Linux client test" | sudo tee /mnt/share/client-linux.txt
```

Проверка:

```
cat /mnt/share/client-linux.txt
ls -l /mnt/share
```

Ожидаемый текст:

```
Linux client test
```

Отключение ресурса:

```
sudo umount /mnt/share
```

### Проверка по имени file01

Если DNS в лаборатории не настроен, на Linux-клиенте можно добавить запись:

```
echo '192.168.10.20 file01' | sudo tee -a /etc/hosts
```

Проверка:

```
getent hosts file01
```

Ожидаемый вывод:

```
192.168.10.20 file01
```

После этого ресурс можно проверить по имени:

```
smbclient -L //file01 -U ivanov
```

---

## 19. Проверка с Windows-клиента

В командной строке Windows проверяется связь:

```
ping 192.168.10.20
```

Старые подключения удаляются, чтобы Windows не использовала сохранённую учётную запись:

```
net use * /delete /y
```

Ресурс подключается как диск Z::

```
net use Z: \\192.168.10.20\share /user:ivanov *
```

Символ \* означает, что пароль будет запрошен отдельно и не останется в истории команды.

Ожидаемый результат:

```
The command completed successfully.
```

Проверка содержимого:

```
dir Z:\
```

Создание файла:

```
echo Windows client test > Z:\client-win.txt
```

Проверка:

```
type Z:\client-win.txt
```

Ожидаемый вывод:

```
Windows client test
```

После проверки подключение удаляется:

```
net use Z: /delete
```

Если ресурс работает по IP-адресу, но не работает как \\file01\share, проблема относится к разрешению имени, а не к Samba.

---

## 20. Дополнительное сравнение: гостевой ресурс только для чтения

Основной ресурс share должен оставаться авторизованным. Для сравнения можно создать отдельный каталог, который не содержит важных данных.

```
sudo mkdir -p /srv/public
sudo chown root:root /srv/public
sudo chmod 0755 /srv/public
echo "Public training file" | sudo tee /srv/public/readme.txt
```

В секции [global] файла /etc/samba/smb.conf добавляется:

```
map to guest = Bad User
```

В конец файла добавляется:

```
[public]
  path = /srv/public
  browsable = yes
  read only = yes
  guest ok = yes
```

Проверка и перезапуск:

```
sudo testparm -s
sudo systemctl restart smbd
```

Проверка без пароля:

```
smbclient //localhost/public -N -c 'ls; get readme.txt /tmp/public-readme.txt'
```

Проверка загруженного файла:

```
cat /tmp/public-readme.txt
```

Ожидаемый вывод:

```
Public training file
```

Современная Windows может блокировать небезопасный гостевой SMB-доступ. Ради учебного теста не нужно ослаблять политику Windows. Достаточно проверить ресурс с Linux-клиента.

---

## Часть 3. Резервное копирование

### 21. Подготовка тестовых данных

Перед созданием backup полезно добавить несколько файлов:

```
sudo -u ivanov bash -c 'echo "Quarterly report" > /srv/share/report.txt'
sudo -u petrova bash -c 'mkdir -p /srv/share/documents'
sudo -u petrova bash -c 'echo "Contract draft" > /srv/share/documents/contract.txt'
```

Проверка:

```
find /srv/share -maxdepth 2 -type f -printf '%p | %u:%g | %m\n'
```

Пример:

```
/srv/share/report.txt | ivanov:office | 660
/srv/share/documents/contract.txt | petrova:office | 660
```

---

### 22. Создание backup-скрипта

Скрипт будет:

- проверять, что /srv/share и /backup смонтированы;
- создавать архив с датой и временем;
- сохранять данные и конфигурацию Samba;
- проверять архив командой tar -tzf;
- создавать контрольную сумму SHA-256;
- вести журнал;
- удалять архивы старше 14 дней;
- блокировать параллельный запуск второго экземпляра.

Файл создаётся:

```
sudo nano /usr/local/sbin/backup-share.sh
```

Содержимое:

```
#!/usr/bin/env bash

set -Eeuo pipefail
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

SOURCE_MOUNT="/srv/share"
BACKUP_MOUNT="/backup"
DEST="/backup/file01"
LOG="/var/log/backup-share.log"
```

```

RETENTION_DAYS=14
STAMP="$(date +%F_%H-%M-%S)"
ARCHIVE_NAME="share-${STAMP}.tar.gz"
ARCHIVE_PATH="${DEST}/${ARCHIVE_NAME}"
CHECKSUM_PATH="${ARCHIVE_PATH}.sha256"

exec 9>/run/lock/backup-share.lock
flock -n 9 || {
    echo "Другой экземпляр backup уже выполняется."
    exit 0
}

exec >>"$LOG" 2>&1

echo "[$(date --iso-8601=seconds)] START"

mountpoint -q "$SOURCE_MOUNT" || {
    echo "ERROR: $SOURCE_MOUNT не является точкой монтирования"
    exit 10
}

mountpoint -q "$BACKUP_MOUNT" || {
    echo "ERROR: $BACKUP_MOUNT не является точкой монтирования"
    exit 11
}

mkdir -p "$DEST"

tar -czf "$ARCHIVE_PATH" -C / \
    srv/share \
    etc/samba/smb.conf

tar -tzf "$ARCHIVE_PATH" >/dev/null

(
    cd "$DEST"
    sha256sum "$ARCHIVE_NAME" > "${ARCHIVE_NAME}.sha256"
)

find "$DEST" -type f -name 'share-*.tar.gz' \
    -mtime +"$RETENTION_DAYS" -delete

find "$DEST" -type f -name 'share-*.tar.gz.sha256' \
    -mtime +"$RETENTION_DAYS" -delete

echo "OK: $ARCHIVE_PATH ($(du -h "$ARCHIVE_PATH" | cut -f1))"
echo "[$(date --iso-8601=seconds)] END"

```

## Разбор ключевых строк

| Строка             | Смысл                                                                    |
|--------------------|--------------------------------------------------------------------------|
| set -Eeuo pipefail | завершает скрипт при большинстве необработанных ошибок                   |
| mountpoint -q      | не даёт создать backup в обычном пустом каталоге, если диск не подключён |

| Строка          | Смысл                                                |
|-----------------|------------------------------------------------------|
| flock           | не допускает одновременный запуск двух копий скрипта |
| tar -czf        | создаёт сжатый архив                                 |
| tar -tzf        | проверяет, что архив читается                        |
| sha256sum       | создаёт контрольную сумму                            |
| find ... -mtime | удаляет старые архивы по сроку хранения              |

Права файла:

```
sudo chown root:root /usr/local/sbin/backup-share.sh
sudo chmod 750 /usr/local/sbin/backup-share.sh
```

Проверка синтаксиса:

```
sudo bash -n /usr/local/sbin/backup-share.sh
```

Если синтаксис правильный, команда ничего не выводит и возвращает код 0.

Код возврата можно проверить:

```
echo $?
```

Ожидаемый вывод:

```
0
```

## 23. Ручной запуск backup

```
sudo /usr/local/sbin/backup-share.sh
```

Скрипт пишет основной вывод в журнал, поэтому в консоли может не появиться ничего.

Проверка журнала:

```
sudo tail -n 20 /var/log/backup-share.log
```

Пример успешного результата:

```
[2026-07-16T13:20:01+05:00] START
OK: /backup/file01/share-2026-07-16_13-20-01.tar.gz (8.0K)
[2026-07-16T13:20:01+05:00] END
```

Проверка файлов:

```
sudo ls -lh /backup/file01
```

Пример:

```
-rw-r--r-- 1 root root 2.3K Jul 16 13:20 share-2026-07-16_13-20-01.tar.gz
-rw-r--r-- 1 root root 99 Jul 16 13:20 share-2026-07-16_13-20-01.tar.gz.sha256
```

Проверка содержимого свежего архива:

```
ARCHIVE=$(ls -lt /backup/file01/share-*.tar.gz | head -n 1)
sudo tar -tzf "$ARCHIVE" | head -n 20
```

Ожидаемые пути:

```
srv/share/  
srv/share/report.txt  
srv/share/documents/  
srv/share/documents/contract.txt  
etc/samba/smb.conf
```

Проверка контрольной суммы:

```
cd /backup/file01  
sudo sha256sum -c "$(basename "$ARCHIVE").sha256"
```

Ожидаемый результат:

```
share-2026-07-16_13-20-01.tar.gz: OK
```

### Самопроверка этапа

Бэкап считается созданным корректно, если одновременно выполняются условия:

- в журнале есть START, OK и END;
- архив существует и не имеет нулевой размер;
- `tar -tzf` выводит содержимое;
- в архиве есть `srv/share` и `etc/samba/smb.conf`;
- `sha256sum -c` возвращает OK.

---

## 24. Настройка запуска через cron

Создаётся отдельный файл расписания:

```
sudo nano /etc/cron.d/backup-share
```

Содержимое:

```
SHELL=/bin/bash  
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin  
30 23 * * * root /usr/local/sbin/backup-share.sh
```

Эта запись запускает бэкап каждый день в 23:30.

Права файла:

```
sudo chown root:root /etc/cron.d/backup-share  
sudo chmod 644 /etc/cron.d/backup-share
```

Служба запускается и добавляется в автозагрузку:

```
sudo systemctl enable --now cron  
sudo systemctl status cron --no-pager
```

Ожидаемая строка:

```
Active: active (running)
```

### Быстрая проверка расписания

Чтобы не ждать 23:30, строка временно меняется на запуск каждые две минуты:

```
*/2 * * * * root /usr/local/sbin/backup-share.sh
```

После сохранения студент ждёт смены двухминутного интервала и проверяет:

```
sudo ls -lt /backup/file01 | head
sudo tail -n 20 /var/log/backup-share.log
```

Для Debian и Ubuntu сообщения cron можно посмотреть так:

```
sudo journalctl -u cron --since "10 minutes ago" --no-pager
```

Если система пишет cron-события в /var/log/syslog, подойдет команда:

```
sudo grep CRON /var/log/syslog | tail -n 20
```

После успешного теста обязательно возвращается ежедневное расписание:

```
30 23 * * * root /usr/local/sbin/backup-share.sh
```

### Самопроверка cron

```
sudo stat -c '%U:%G %a %n' /etc/cron.d/backup-share
sudo systemctl is-active cron
```

Ожидаемый результат:

```
root:root 644 /etc/cron.d/backup-share
active
```

---

## 25. Проверка восстановления файла

Наличие архива ещё не доказывает, что backup рабочий. Для проверки файл нужно удалить и восстановить.

### 25.1. Создание контрольного файла

```
echo "restore test $(date --iso-8601=seconds)" | sudo tee /srv/share/restore-control.txt
sudo chown root:office /srv/share/restore-control.txt
sudo chmod 0660 /srv/share/restore-control.txt
```

Проверка содержимого:

```
cat /srv/share/restore-control.txt
```

### 25.2. Создание свежей копии

```
sudo /usr/local/sbin/backup-share.sh
```

Определяется самый свежий архив:

```
ARCHIVE=$(ls -lt /backup/file01/share-*.tar.gz | head -n 1)
echo "$ARCHIVE"
```

Пример:

```
/backup/file01/share-2026-07-16_13-42-10.tar.gz
```

Студент убеждается, что файл действительно попал в архив:

```
sudo tar -tzf "$ARCHIVE" | grep 'srv/share/restore-control.txt'
```

Ожидаемый вывод:

```
srv/share/restore-control.txt
```

### 25.3. Удаление исходного файла

```
sudo rm /srv/share/restore-control.txt
```

Проверка:

```
ls /srv/share/restore-control.txt
```

Ожидаемая ошибка:

```
ls: cannot access '/srv/share/restore-control.txt': No such file or directory
```

### 25.4. Восстановление в отдельный каталог

```
sudo rm -rf /tmp/restore-test
sudo mkdir -p /tmp/restore-test
sudo tar -xzf "$ARCHIVE" -C /tmp/restore-test \
    srv/share/restore-control.txt
```

Проверка содержимого:

```
sudo cat /tmp/restore-test/srv/share/restore-control.txt
```

Должен появиться исходный текст restore test ....

### 25.5. Возврат файла в общий ресурс

```
sudo install -o root -g office -m 0660 \
    /tmp/restore-test/srv/share/restore-control.txt \
    /srv/share/restore-control.txt
```

Проверка:

```
ls -l /srv/share/restore-control.txt
cat /srv/share/restore-control.txt
```

Пример прав:

```
-rw-rw---- 1 root office ... /srv/share/restore-control.txt
```

После этого файл проверяется через Samba с Linux- или Windows-клиента.

### Самопроверка восстановления

Восстановление считается успешным, если:

- удалённый файл извлечён из архива;
  - его содержимое совпадает с исходным;
  - владелец и права позволяют работать с ним через Samba;
  - файл виден с клиентской машины.
-



## Часть 4. Проверка отказоустойчивости RAID1

### 26. Учебная симуляция отказа одного диска

Этот раздел выполняется только в VirtualBox после создания снимка. Перед началом полезно убедиться, что свежий backup уже создан.

Текущее состояние:

```
cat /proc/mdstat
```

Ожидается [UU].

Один диск помечается как неисправный:

```
sudo mdadm /dev/md0 --fail /dev/sdc
```

Диск удаляется из массива:

```
sudo mdadm /dev/md0 --remove /dev/sdc
```

Проверка:

```
cat /proc/mdstat  
sudo mdadm --detail /dev/md0
```

Пример состояния:

```
md0 : active raid1 sdb[0]  
... [2/1] [U_]
```

В подробном выводе будет:

```
State : clean, degraded  
Active Devices : 1  
Failed Devices : 0  
Spare Devices : 0
```

Состояние degraded означает, что данные доступны, но резервного зеркала больше нет.

#### Проверка доступности данных

```
findmnt /srv/share  
cat /srv/share/report.txt
```

С клиента создаётся файл через Samba. Например, с Linux-клиента:

```
smbclient //192.168.10.20/share -U ivanov \  
-c 'put /etc/hostname degraded-test.txt; ls'
```

Если файл создаётся, RAID1 действительно продолжает обслуживать данные с одного диска.

Это не нормальный режим для постоянной работы. Пока массив degraded, отказ оставшегося диска может привести к потере хранилища.

---

### 27. Возврат диска в RAID1

На учебном стенде тот же виртуальный диск можно очистить от старой RAID-сигнатуры и добавить обратно.

```
sudo mdadm --zero-superblock --force /dev/sdc  
sudo mdadm /dev/md0 --add /dev/sdc
```

Проверка синхронизации:

```
watch -n 2 cat /proc/mdstat
```

Пример:

```
md0 : active raid1 sdc[2] sdb[0]
    ... [2/1] [U_]
    [====>.....] recovery = 28.0%
```

После завершения:

```
cat /proc/mdstat
sudo mdadm --detail /dev/md0
```

Ожидаемый результат:

```
[UU]
State : clean
Active Devices : 2
Failed Devices : 0
```

Если mdadm --add принимает диск без --zero-superblock, очистку сигнатуры можно не выполнять.

## Часть 5. Диагностика

### 28. Как искать ошибку по уровням

У файлового сервера несколько уровней. Проверять их удобнее снизу вверх:

```
виртуальный диск
↓
RAID1
↓
файловая система
↓
монтирование
↓
Linux-права
↓
Samba
↓
сеть и учётные данные клиента
```

Если нижний уровень не работает, настройка верхнего уровня не поможет. Например, разрешение записи в smb.conf не исправит запрет на уровне прав каталога.

### 29. Типовые неисправности

| Симптом                             | Что проверить                            | Команды                                                          |
|-------------------------------------|------------------------------------------|------------------------------------------------------------------|
| /dev/md0 не появился после загрузки | конфигурацию и сборку массива            | cat /proc/mdstat, mdadm --assemble --scan, mdadm --detail --scan |
| RAID имеет [U_]                     | один участник отсутствует или неисправен | mdadm --detail /dev/md0, lsblk                                   |
| mount -a сообщает ошибку            | UUID, путь, тип файловой системы         | blkid, findmnt --verify --verbose, cat /etc/fstab                |

| Симптом                                                                                     | Что проверить                                                                                                       | Команды                                                                                                         |
|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| после монтирования «пропали» файлы<br>testparm показывает ошибку<br>NT_STATUS_LOGON_FAILURE | файлы были созданы в каталоге до подключения FS<br>синтаксис smb.conf<br>нет Samba-пользователя или неверный пароль | umount /srv/share, затем ls -la /srv/share<br>testparm -s<br>pdbedit -L, smbpasswd -a ivanov                    |
| NT_STATUS_ACCESS_DENIED                                                                     | группа, ACL или valid users                                                                                         | id ivanov, getent group office, getfacl /srv/share<br>getent hosts file01, /etc/hosts, DNS                      |
| по IP работает, по имени нет                                                                | разрешение имени                                                                                                    | net use * /delete /y                                                                                            |
| Windows использует другого пользователя<br>скрипт работает вручную, но не из cron           | сохранённое SMB-подключение<br>права, формат cron, служба                                                           | systemctl status cron, journalctl -u cron, stat /etc/cron.d/backup-share<br>mountpoint /backup, findmnt /backup |
| backup создаётся в обычном каталоге<br>архив повреждён                                      | /backup не смонтирован<br>ошибка записи или неполный файл                                                           | tar -tzf архив, sha256sum -c файл.sha256                                                                        |

### Полезный набор команд для быстрой диагностики

```
lsblk -f
cat /proc/mdstat
sudo mdadm --detail /dev/md0
findmnt --target /srv/share
findmnt --target /backup
ls -ld /srv/share
getfacl /srv/share
sudo testparm -s
sudo systemctl status smbd --no-pager
sudo ss -lntp | grep ':445'
sudo pdbedit -L
sudo tail -n 30 /var/log/backup-share.log
```

## Часть 6. Итоговая самостоятельная проверка

### 30. Чек-лист готовности сервера

Студент последовательно выполняет проверки и отмечает результат.

#### Хранилище

- ☐ lsblk показывает два диска в составе /dev/md0.
- ☐ cat /proc/mdstat показывает [UU].
- ☐ mdadm --detail /dev/md0 показывает raid1, два активных устройства и ноль failed-устройств.
- ☐ /srv/share смонтирован с /dev/md0.
- ☐ /backup смонтирован с отдельного диска.
- ☐ findmnt --verify --verbose не сообщает ошибок.
- ☐ После перезагрузки обе файловые системы подключаются автоматически.

## Права и Samba

- ☐ Каталог /srv/share принадлежит root:office.
- ☐ Режим каталога содержит setgid: drwxrws---.
- ☐ Пользователи ivanov и petrova входят в office.
- ☐ pdbedit -L показывает обоих пользователей.
- ☐ testparm -s загружает конфигурацию без ошибок.
- ☐ systemctl is-active smbd возвращает active.
- ☐ TCP-порт 445 прослушивается.
- ☐ Файл, созданный через Samba, получает группу office.
- ☐ Linux-клиент может читать и записывать файлы.
- ☐ Windows-клиент может читать и записывать файлы либо проверка полностью выполнена через smbclient.

## Бэкап и восстановление

- ☐ Скрипт проходит bash -n.
- ☐ Ручной запуск создаёт архив и .sha256.
- ☐ tar -tzf показывает содержимое архива.
- ☐ sha256sum -с возвращает OK.
- ☐ Служба cron активна.
- ☐ Тестовый запуск cron создаёт новый архив.
- ☐ Удалённый файл восстановлен из backup.
- ☐ Восстановленный файл доступен через Samba.

---

## 31. Короткий набор команд итоговой проверки

```
printf '\n=== RAID ===\n'
cat /proc/mdstat
sudo mdadm --detail /dev/md0 | grep -E 'Raid Level|State|Active Devices|Failed Devices'

printf '\n=== MOUNTS ===\n'
findmnt --target /srv/share
findmnt --target /backup
sudo findmnt --verify --verbose

printf '\n=== RIGHTS ===\n'
ls -ld /srv/share
getent group office
getfacl -p /srv/share | sed -n '1,15p'

printf '\n=== SAMBA ===\n'
sudo testparm -s 2>/dev/null | grep -A12 '^\[share\]'
systemctl is-active smbd
sudo ss -lnt | grep ':445'

printf '\n=== BACKUP ===\n'
sudo tail -n 5 /var/log/backup-share.log
sudo ls -lh /backup/file01 | tail
```

Этот блок не заменяет проверки с клиентов и пробное восстановление, но быстро показывает основное состояние сервера.

---

## Часть 7. Отчёт по работе

### 32. Что фиксируется в отчёте

Отчёт можно оформить в Markdown. В нём достаточно оставить только фактические результаты, а не копировать всю методичку.

Рекомендуемая структура:

```
# Отчёт: файловое хранилище и резервное копирование
```

```
## 1. Учебный стенд
```

- ОС сервера:
- IP-адрес сервера:
- Системный диск:
- Диски RAID1:
- Backup-диск:

```
## 2. RAID1
```

Команда создания:

```
```bash
...
```
```

Состояние массива:

```
```text
...
```
```

```
## 3. Файловые системы и автомонтирование
```

UUID RAID:

UUID backup-диска:

Строки `/etc/fstab`:

```
```fstab
...
```
```

```
## 4. Права и Samba
```

Группа доступа:

Пользователи:

Секция `[share]`:

```
```ini
...
```
```

Результат проверки с клиента:

```
```text
...
```
```

## ## 5. Резервное копирование

Имя созданного архива:  
Результат проверки SHA-256:  
Расписание cron:

## ## 6. Восстановление

Какой файл был удалён:  
Из какого архива он восстановлен:  
Результат проверки:

## ## 7. Возникшая проблема

Симптом:  
Причина:  
Команды диагностики:  
Как проблема была устранена:

## ## 8. Вывод

Кратко объясняется различие между RAID1 и backup.

---

### 33. Вопросы для самопроверки

1. Почему два диска по 10 ГБ в RAID1 дают примерно 10 ГБ полезного места, а не 20 ГБ?
2. Что означают [UU] и [U\_] в /proc/mdstat?
3. Почему удаление файла повторяется на обоих дисках RAID1?
4. Почему резервная копия хранится на отдельном виртуальном диске?
5. Чем UUID удобнее имени /dev/sdb в /etc/fstab?
6. Зачем выполнять findmnt --verify и mount -a до перезагрузки?
7. Как setgid влияет на группу новых файлов в /srv/share?
8. Почему valid users = @office не заменяет Linux-права?
9. Зачем Samba-пользователю сначала нужна учётная запись Linux?
10. Что проверяется командой testparm -s?
11. Как определить, что Samba прослушивает стандартный порт SMB?
12. Почему наличие архива ещё не доказывает работоспособность backup?
13. Что даёт контрольная сумма SHA-256?
14. Почему backup-скрипт проверяет точки монтирования до запуска tar?
15. Какой результат должен дать полноценный тест восстановления?

---

### 34. Итог

После выполнения работы на сервере file01 должно получиться хранилище со следующей цепочкой:

```
два виртуальных диска
↓
RAID1 /dev/md0
↓
ext4 и автмонтирование по UUID
↓
```

```
/srv/share с группой office
↓
авторизованный ресурс Samba
↓
клиенты Linux и Windows
```

Отдельно работает цепочка восстановления:

```
/srv/share + /etc/samba/smb.conf
↓
backup-скрипт
↓
архив на отдельном диске /backup
↓
проверка tar + SHA-256
↓
пробное восстановление файла
```

RAID1 и backup в этой схеме дополняют друг друга: первый поддерживает доступность при отказе диска, второй возвращает данные после удаления, повреждения или ошибки администратора.